PathCare Labs SaaS

Master Execution Blueprint — Solo Developer Edition

Phase 1: 16-Sprint MVP Plan | Month 1 Technical Deep Dive

Prepared for: Solo Full-Stack Developer (NestJS + Next.js)

Stack: NestJS · Next.js 14 · PostgreSQL · Redis · AWS · TypeORM

1. System Architecture — Ek Nazar Mein

PathCare Labs ek multi-tenant SaaS platform hai — matlab ek hi codebase pe kai alag labs run hongi. Har lab ka data ek dusre se bilkul alag rahega. Niche poora system 4 layers mein explain kiya gaya hai:

1.1 Frontend Layer — Next.js 14

Jo user browser mein dekhta hai. Yeh layer sirf UI render karti hai aur backend APIs se data fetch karti hai.

- Next.js 14 App Router — server-side rendering + client components dono
- shadcn/ui component library — production-ready accessible components
- PWA (Progressive Web App) — offline kaam karne ki capability, IndexedDB storage
- Storybook — component documentation aur design system
- Auth pages: Login, Forgot Password, Role-based Dashboard views

1.2 Backend Layer — NestJS (Node.js)

Poori business logic yahan hoti hai. Har incoming request is server se guzarti hai.

- NestJS modular architecture — har feature ka apna module, controller, service
- Multi-tenancy middleware — subdomain se tenant identify karna
- JWT Authentication — access token (15 min) + refresh token (7 days)
- RBAC Guards — role-based access control, 5 roles
- Audit Log Interceptor — har sensitive action automatically log hota hai
- BullMQ Queue Processor — background jobs (PDF generation, emails)
- Socket.io — real-time notifications aur status updates

1.3 Database Layer — PostgreSQL (AWS RDS)

Schema-per-tenant approach use ki gayi hai. Ek database ke andar kai schemas hote hain — har lab ka apna isolated schema.

- public schema — shared data: tenants table, global config
- tenant_[labname] schema — har lab ke liye alag: users, patients, tests, reports
- TypeORM migrations — code se database changes manage karna
- Har request mein SET search_path = tenant_[name] execute hota hai

1.4 Supporting Services Layer

Background mein kaam karne wale tools jo user directly nahi dekhta:

- Redis (AWS ElastiCache) — session cache, rate limiting counters, BullMQ queue backend
- AWS S3 — lab reports PDF, images, logos ka unlimited storage
- AWS Secrets Manager — DB passwords, JWT secrets, API keys secure storage
- AWS Route53 + ACM — wildcard subdomain routing (*.pathcare.com) aur SSL
- GitHub Actions CI/CD — har PR pe automatic lint + build + deploy

1.5 RBAC — 5 User Roles

Har user ke paas exactly ek role hoga jo decide karega wo kya dekh aur kar sakta hai:

Role	Access Level	Key Permissions
Super Admin	Global	Saare labs, billing, tenant management, system config
Lab Admin	Per-Lab	Apni lab ka poora control — staff, settings, reports, rates
Receptionist	Per-Lab	Patient registration, sample collection, billing, receipts
Lab Technician	Per-Lab	Test results enter karna, sample processing, QC
Doctor / Patient	Read-Only	Sirf apne reports dekhna, download karna

2. Multi-Tenancy — Schema-per-Tenant Explained

Yeh concept poore project ki neenv hai. Ek baar isko properly samajh lo toh baaki sab clear ho jayega.

2.1 Multi-tenancy Kya Hota Hai?

Socho ek building hai jisme kai alag offices hain — building ek hai, lekin har office ka apna darwaza, apna lock, apna data hai. Waise hi ek PostgreSQL database mein kai schemas hote hain.

Real PathCare example:

- metropolis.pathcare.com ➜ PostgreSQL schema: tenant_metropolis
- thyrocare.pathcare.com ➜ PostgreSQL schema: tenant_thyrocare
- lalpath.pathcare.com ➜ PostgreSQL schema: tenant_lalpath

Metropolis waale kabhi Thyrocare ka data nahi dekh sakte — schemas alag hain.

2.2 Request Flow — Subdomain se Schema Tak

Jab koi browser se request aati hai, yeh steps hote hain:

- Step 1: Browser request bhejta hai — metropolis.pathcare.com/api/patients
- Step 2: NestJS middleware subdomain nikalta hai: 'metropolis'
- Step 3: Public schema mein dhundta hai — metropolis ka schema name kya hai
- Step 4: PostgreSQL ko command deta hai: SET search_path = tenant_metropolis
- Step 5: Ab saari queries sirf us lab ke data pe chalti hain

2.3 Schema Approach Comparison

Approach	Data Isolation	Complexity	Cost	Our Choice
Schema per tenant	Strong	Medium	Low (1 DB)	YES
Row-level (tenant_id column)	Weak	Low	Very Low	NO
Separate DB per tenant	Strongest	High	Very High	NO
Hybrid (large = own DB)	Strong	Very High	High	NO

3. AWS Setup Guide — Pehli Baar ke Liye

AWS bilkul naya hai toh pehle samjho ki kaunsi service kya kaam karti hai, phir setup karo.

3.1 AWS Services — Simple Hindi Mein

AWS Service	Kya Hai	PathCare Mein Use
RDS (PostgreSQL)	Managed database server — apne local PostgreSQL jaisa lekin AWS pe	Har tenant ka data, user accounts, test results sab yahan
ElastiCache (Redis)	Ultra-fast in-memory storage — RAM pe data rakho	Session cache, rate limiting, BullMQ queue, Socket.io pub/sub
S3 Bucket	Unlimited file storage — Google Drive jaisa samjho	Lab reports PDF, patient documents, lab logos, exports
Secrets Manager	Encrypted password/key vault — .env se zyada secure	DB password, JWT secret, third-party API keys
Route53	DNS management — domain name ko IP se map karo	pathcare.com aur *.pathcare.com wildcard subdomain
ACM (SSL)	Free HTTPS certificate provider	Har subdomain pe https:// enable karna — mandatory
VPC	Virtual Private Cloud — private network container	RDS aur Redis public internet pe accessible nahi honge

3.2 W1 AWS Setup — Correct Sequence

Yeh order follow karo — ek cheez dusre pe depend karti hai:

- Step 1: AWS Account banao + MFA enable karo (ZARURI — security ke liye)
  - Free tier use karo — RDS t3.micro, ElastiCache t3.micro pehle 12 mahine free
  - Region: ap-south-1 (Mumbai) — India ke users ke liye best latency
- Step 2: VPC setup — ek private network
  - Public aur private subnets banao — RDS/Redis private mein rahenge
  - Security Groups: only app server ko RDS/Redis access allow karo
- Step 3: S3 bucket banao
  - Bucket name globally unique hona chahiye, e.g. pathcare-files-prod
  - Block all public access ON rakho — files private rahein
- Step 4: Secrets Manager entries
  - Abhi placeholders daalo: db_password, jwt_secret — real values baad mein
- Step 5: RDS PostgreSQL create karo

    - VPC ke andar, private subnet mein. Public access: OFF
    - db.t3.micro — free tier eligible. PostgreSQL 15+
- Step 6: ElastiCache Redis banao
    - Same VPC mein. cache.t3.micro — free tier eligible
- Step 7: Route53 + ACM SSL
    - Wildcard certificate request karo: *.pathcare.com
    - DNS validation se certificate milta hai — usually 5-10 min

4. Month 1 — Sprint Breakdown (Week 1-4)

Har sprint 1 hafta hai. Solo developer ke liye estimated time ~4-6 hours/day. Total Month 1 = approx 110-130 hours.

Week 1 — Infrastructure Zero

Priority: [BLOCKER] Est. time: 30-35 hours

Yeh week bina kiye aage kuch nahi chalega. Monorepo + AWS ka foundation set hoga.

Day	Task	Details	Tool/Tech
D1	GitHub repo + Turborepo init	apps/api (NestJS), apps/web (Next.js), packages/shared folder structure	Turborepo, Git
D2	Docker Compose — local dev	PostgreSQL + Redis containers locally. AWS nahi — pehle local pe kaam karo	Docker Desktop
D3	Code quality tools setup	ESLint + Prettier + Husky hooks. Commit se pehle format check automatic	ESLint, Prettier
D4	AWS account + S3 + Secrets Manager	MFA zarur lagao. ap-south-1 region. Sirf basic services abhi	AWS Console
D5	GitHub Actions CI pipeline	PR pe: lint check + type check + build. Sirf fail-fast pipeline, deploy baad mein	GitHub Actions

W1 Key Decisions

- AWS pe W1 mein sirf S3 aur Secrets Manager banao — RDS aur Redis W1 ke end mein ya W2 start mein
- Turborepo ke liye sirf 2 apps rakho: api aur web. Packages abhi mat banao — premature abstraction avoid karo
- CI pipeline mein deploy abhi mat lagao — sirf lint + build check. Deployment W3 mein jab auth stable ho

Week 2 — App Skeleton + Multi-tenancy

Priority: [BLOCKER] Est. time: 25-30 hours

Poore project ka sabse complex part yahan hai — multi-tenant middleware likhna. Isko time do, rush mat karo.

Day	Task	Details	Tool/Tech
D1	NestJS app + module structure	AppModule, ConfigModule, DatabaseModule. TypeORM + PostgreSQL connection	NestJS, TypeORM
D2	Tenant resolver middleware — CORE	Subdomain extract → DB lookup → SET search_path. Yeh W2 ka sabse important kaam	NestJS Middleware
D3	TypeORM migrations system	Har tenant schema ke liye alag migrations. Public schema mein tenants table	TypeORM CLI
D4	Next.js 14 + shadcn/ui setup	App Router, basic layout, color tokens. Storybook is week optional hai	Next.js 14, shadcn
D5	End-to-end tenant routing test	2 fake tenants locally banao, verify karo routing sahi kaam kar rahi hai	Postman/curl

W2 Key Decisions

- Tenant middleware mein error handling zarur karo: unknown subdomain = 404, not crash
- TypeORM schema switching ka pattern ek baar sahi se likho — yeh pattern poore project mein repeat hoga
- Storybook agar time kam pade toh skip karo — MVP ke baad add karo

Week 3 — Authentication System

Priority: [CRITICAL] Est. time: 20-25 hours

NestJS experience hai toh yeh week comfortable rahega. JWT + RBAC patterns familiar honge.

Day	Task	Details	Tool/Tech
D1	Argon2id hashing + User entity	bcrypt nahi — Argon2id use karo (zyada secure). User table har tenant schema mein	argon2 npm pkg
D2	JWT access + refresh tokens	Access: 15 min. Refresh: 7 days. Refresh token DB mein store karo (revocation ke liye)	NestJS JWT
D3	Rate limiting + RBAC guards	5 failed login = 15 min Redis block. @Roles() decorator. Guards NestJS pe	Redis, NestJS Guards
D4	Audit log interceptor	Har sensitive action auto-log: tenant, user, action, timestamp, IP	NestJS Interceptor
D5	Frontend: Login page + cookie storage	httpOnly cookie mein JWT store karo — localStorage nahi. XSS se protection	Next.js, cookies

W3 Key Decisions

- JWT httpOnly cookie — localStorage kabhi mat use karo, XSS attack se tokens chori ho sakte hain
- Refresh token rotation implement karo — jab refresh use ho, naya refresh token issue karo aur old invalidate karo
- Rate limiting Redis mein rakho — restart pe counters reset nahi honge

Week 4 — Masters Data + Queue System

Priority: **[HIGH]** Est. time: 25-30 hours

BullMQ aur Socket.io probably naye honge — pehle docs padho, phir implement karo.

Day	Task	Details	Tool/Tech
D1	DB schema: Lab/Dept/Sample/Employee	Tables design karo. Yeh CRUD screens pattern establish karne ke baad AI/Claude se bhi generate ho sakti hain	TypeORM entities
D2	BullMQ setup Redis ke saath	Background jobs: report generate, email send. User request block nahi hoga	BullMQ, Redis
D3	Socket.io real-time events	Test ready notification, sample status update. Multi-tenant rooms implement karo	Socket.io, NestJS
D4	Service Worker + IndexedDB (PWA skeleton)	Offline foundation. Abhi sirf basic — full offline support Month 2 mein	Workbox, IndexedDB
D5	Month 1 review + AWS staging deploy	Kya complete hua assess karo. Dev environment AWS pe deploy karo	AWS EC2/ECS

5. Solo Developer Strategy

Yeh plan originally team ke liye design hua tha. Akele execute karne ke liye 5 smart adjustments karo:

Rule 1 — AWS pe W3 mein jao, W1 mein nahi

W1-W2 mein sab kuch Docker Compose pe locally karo. AWS setup time-consuming hai aur agar pehli baar use kar rahe ho toh problems aayenge. W3 mein jab auth stable ho tab staging pe deploy karo. Yeh approach 8-10 hours bachayega W1 mein.

Rule 2 — Turborepo Simple Rakho

Sirf 2 apps banao: apps/api aur apps/web. packages/ folder abhi mat banao. Premature abstraction solo developer ke liye time waste hai. Shared code directly import karo — refactoring baad mein.

Rule 3 — CRUD Screens AI se Generate Karo

W4 mein 'Delegate' likha tha — junior dev nahi hai toh Claude ya GitHub Copilot use karo. Pattern ek baar sahi se likho (API route + DTO + service + controller), phir CRUD screens ek din mein 5-6 modules ki AI se generate ho sakti hain. Bas customize karo.

Rule 4 — Storybook Skip Karo (Abhi)

W2 mein Storybook plan tha — yeh optional hai learning project ke liye. Sirf shadcn/ui directly use karo. Storybook MVP launch ke baad add karo jab design system stable ho jaye.

Rule 5 — Daily 5-Min Standup Apne Saath

Har raat likhlo: (1) Aaj kya complete hua, (2) Kal kya karunga, (3) Kya block hai. GitHub Project Board pe Kanban use karo — To Do, In Progress, Done. Solo dev ke liye yeh discipline MVP tak pahunchne ka sabse bada factor hai.

5.1 Realistic Time Estimates — Solo Developer

Week	Focus Area	Estimated Hours	Risk Level	Blocker?
Week 1	Infrastructure + AWS (naya)	30-35 hrs	**HIGH — AWS pehli baar**	YES
Week 2	Multi-tenancy middleware	25-30 hrs	**HIGH — complex logic**	YES
Week 3	Auth system (JWT + RBAC)	20-25 hrs	**MEDIUM — NestJS familiar hai**	NO
Week 4	Masters + BullMQ + Socket.io	25-30 hrs	**MEDIUM — new tools**	NO

5.2 Week 1 Daily Schedule (Recommended)

Time Slot	Activity	Hours
9:00 - 9:15	Daily standup — review GitHub board	0.25
9:15 - 12:00	Deep work — main coding task (no distractions)	2.75
12:00 - 13:00	Lunch break	-
13:00 - 15:30	Deep work — continue / second task	2.5
15:30 - 16:00	Documentation / commit / PR notes	0.5
16:00 - 16:15	Tomorrow ka plan likhna	0.25
	Total productive coding time	~6 hrs/day

6. Quick Reference — Commands & Checklist

6.1 Initial Setup Commands

W1 D1 ke liye yeh commands sequence mein run karo:

```
# 1. Turborepo monorepo initialize karo
npx create-turbo@latest pathcare-saas

# 2. NestJS app banao apps/ folder mein
cd apps && nest new api --package-manager npm

# 3. Next.js 14 app banao
npx create-next-app@latest web --typescript --tailwind --app

# 4. Key NestJS packages install karo
npm i @nestjs/typeorm typeorm pg @nestjs/jwt @nestjs/passport passport passport-jwt
argon2 bull @nestjs/bull ioredis socket.io @nestjs/websockets
```

6.2 Month 1 Completion Checklist

#	Milestone	Week	Status
1	Monorepo setup with Turborepo + CI pipeline	W1	Pending
2	Docker Compose local dev environment working	W1	Pending
3	AWS account + S3 + Secrets Manager configured	W1	Pending
4	NestJS app with TypeORM + PostgreSQL connection	W2	Pending
5	Multi-tenant middleware working end-to-end	W2	Pending
6	Tenant schema isolation verified with 2 test tenants	W2	Pending
7	Next.js 14 + shadcn/ui design system setup	W2	Pending
8	JWT auth with access + refresh token rotation	W3	Pending
9	Argon2id password hashing implemented	W3	Pending
10	Rate limiting (5 fails = 15 min block) working	W3	Pending
11	RBAC guards with all 5 roles functional	W3	Pending
12	Audit log interceptor logging sensitive actions	W3	Pending
13	Login UI with httpOnly cookie JWT storage	W3	Pending
14	Lab/Dept/Sample/Employee DB schemas created	W4	Pending
15	BullMQ + Redis queue processing background jobs	W4	Pending
16	Socket.io real-time events per tenant	W4	Pending

#	Milestone	Week	Status
17	PWA Service Worker skeleton	W4	Pending
18	Dev environment deployed to AWS staging	W4	Pending